A

Project Report on

JOB CHAINING AND BUILD PIPELINES

Submitted for partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

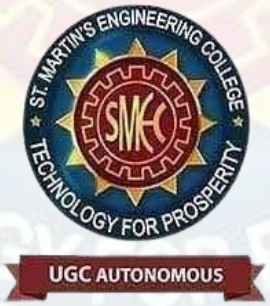
by

YANDRA MOUNIKA- 24K81A05R5

Under the Guidance of

Mr.M.Dileep Kumar

Assistant Professor



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

St. MARTIN'S ENGINEERING COLLEGE

UGC Autonomous

Affiliated to JNTUH, Approved by AICTE Accredited

by NBA & NAAC A+, ISO 9001-2008 Certified.

Dhulapally, Secunderabad-500 100

www.smec.ac.in

NOVEMBER- 2026

St. MARTIN'S ENGINEERING COLLEGE

UGC Autonomous

Affiliated to JNTUH, Approved by AICTE Accredited

by NBA & NAAC A+, ISO 9001-2008 Certified.

Dhulapally, Secunderabad-500 100

www.smec.ac.in

CERTIFICATE

This is to certify that the project titled **"Job Chaining and Build Pipelines"** is being submitted by YANDRA MOUNIKA(24K81A05R5) in fulfillment of the requirement for the award of degree of **BACHELOR OF TECHNOLOGY IN COMPUTER SCIENCE AND ENGINEERING** is recorded of bonafide work carried out by them. The result embodied in this report have been verified and found satisfactory.

Signature of Guide

Mr. M.Dileep Kumar

Assistant Professor

Department of CSE

Signature of HOD

Mr. D. Krishna Kishore

Associate Professor & HOD

Department of CSE

Internal Examiner **External Examiner**

St. MARTIN'S ENGINEERING COLLEGE

UGC Autonomous

Affiliated to JNTUH, Approved by AICTE Accredited

by NBA & NAAC A+, ISO 9001-2008 Certified.

Dhulapally, Secunderabad-500 100

www.smec.ac.in

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

DECLARATION

I, the student of **"Bachelor of Technology in Department of Computer Science & Engineering"**, session: 2024 - 2028**, St. Martin's Engineering College**, Dhulapally, Kompally, Secunderabad, hereby declare that the work presented in this project work entitled **"Job Chaining and Build Pipelines"** is the outcome of my own bonafide work and is correct to the best of my knowledge and this work has been undertaken taking care of Engineering Ethics. This result embodied in this project report has not been submitted in any university for award of any degree.

YANDRA MOUNIKA (24K81A05R5)

Acknowledgement

The satisfaction and euphoria that accompanies the successful completion of any task would be incomplete without the mention of the people who made it possible and whose encouragement and guidance have crowded our efforts with success.

First and foremost, I would like to express our deep sense of gratitude and indebtedness to our College Management for their kind support and permission to use the facilities available in the Institute.

I especially would like to express our deep sense of gratitude and indebtedness to **Prof. Dr. K. RAVINDRA**, Director & Principal, St. Martin's Engineering collage Dhulapally, for permitting us to undertake this project.

I am also thankful to, **Mr.D. KRISHNA KISHORE** Head of the Department, Computer science and engineering, St. Martin's Engineering College, Dhulapally, Secunderabad, for his support and guidance throughout our project as well as Project Coordinator **Mr.M.Dileep Kumar**, Assistant Professor, Department of Computer science and Engineering for her valuable support.

I would like to express our sincere gratitude and indebtedness to our project supervisor **Mr.M.Dileep Kumar**, Assistant Professor, Computer Science and Engineering, St. Martin's Engineering College, Dhulapally, for his support and guidance throughout our project.

Finally, I express thanks to all those who have helped us successfully completing this project. Furthermore, I would like to thank my family and friends for their moral support and encouragement. I express thanks to all those who have helped us in successfully completing the project.

YANDRA MOUNIKA (24K81A05R5)

CONTENTS

1. ABTRACT
2. INTRODUCTION
3. SYSTEM ANALYSIS
4. SYSTEM REQUIREMENTS
5. ALGORITHM
6. SYSTEM IMPLEMENTATION
7. CONCLUSION
8. FUTURE ENHANCEMENTS
9. REFERENCES

1. ABSTRACT

The **Job chaining and build pipelines** are essential concepts in Continuous Integration and Continuous Delivery, which help automate the software development and deployment process. Job chaining refers to connecting multiple jobs so that they execute in a specific sequence based on dependencies or conditions. Build pipelines organize various stages of software development. These concepts reduce manual intervention, improve development efficiency, minimize human errors, and ensure consistent software delivery. Tools such as Jenkins, GitHub Actions, GitLab CI/CD, and Azure DevOps provide features to configure and manage job dependencies and build pipelines. By integrating these tools into the development lifecycle, organizations can detect errors early, maintain software quality, accelerate release cycles, and improve collaboration among development and operations teams. The primary objective of job chaining and build pipelines is to establish a reliable, automated, and efficient process for building, testing, and deploying software applications. This approach supports modern DevOps practices and enables organizations to deliver high-quality software more frequently and consistently.

TECHNOLOGY FOR PROSPERITY
UGC AUTONOMOUS

2. INTRODUCTION

Job chaining and build pipelines are important components of CI/CD that help automate and organize software development activities. Job chaining is the process of linking multiple jobs so that one job triggers another based on predefined dependencies or execution conditions. For example, a build job can trigger a testing job after successful compilation, followed by a deployment job when all tests pass. This approach ensures that tasks are performed in the correct sequence and reduces the need for manual intervention. In modern software development, delivering high-quality software quickly and reliably is an important goal for organizations. Traditional SD processes often involve manual tasks and these manual processes can be time-consuming, error-prone, and difficult to manage as projects become more complex. To overcome these challenges, automation has become an essential part of software development through CI/CD. A build pipeline is an automated workflow that consists of multiple stages, including source code retrieval, compilation, testing, code quality analysis, artifact creation, and deployment. It provides a structured way to manage the complete software delivery process, from code changes to application release. Build pipelines also help developers identify errors early, maintain software quality, and ensure consistent results across different environments. Tools such as Jenkins, GitHub Actions, GitLab CI/CD, and Azure DevOps support the implementation of job chaining and build pipelines. These tools enable teams to configure workflows, manage job dependencies, monitor execution, and automate repetitive tasks. By using job chaining and build pipelines, organizations can improve productivity, reduce deployment time, enhance collaboration between development and operations teams, and achieve reliable software delivery. Therefore, job chaining and build pipelines play a significant role in modern DevOps practices by providing an efficient, scalable, and automated approach to software development, testing, and deployment.

3. SYSTEM ANALYSIS

EXISTING SYSTEM :

The existing system for SD and deployment often relies on manual execution of build, and deployment tasks or uses basic automation methods with limited integration between jobs.

Core Architecture and Functional Modules

1. **Source Code Management Module:** Retrieves source code from repositories such as Git.
2. **Build Module:** Compiles source code and generates executable files or application packages.
3. **Job Scheduling Module:** Executes jobs in a predefined sequence based on triggers and dependencies.
4. **Testing Module:** Runs automated tests to verify application functionality.

Key Functional Capabilities:

- **Basic Build Automation:** Performs compilation and packaging of applications.
- **Sequential Job Execution:** Executes jobs in a predefined order.

Major Limitations:

1. High Manual Intervention: Developers may need to trigger and monitor individual jobs manually.
2. Increased Execution Time: Jobs may run sequentially when some tasks could execute in parallel.
3. Delayed Error Detection: Failures in one stage may not be communicated to dependent jobs.

PROPOSED SYSTEM:

The proposed system is an automated job chaining and build pipeline management system designed to improve the efficiency and consistency of software development and deployment.

Key Enhancements

- **Automated Job Chaining:** Automatically triggers subsequent jobs.
- **Integrated Build Pipelines:** Combines build, testing, validation, and deployment stages into a single workflow.
- **Dependency-Based Execution:** Ensures that jobs execute according to their configured dependencies and conditions.

Advantages

- **Reduced Manual Effort:** Automates repetitive build, testing, and deployment activities.
- **Faster Software Delivery:** Uses automated execution and parallel processing.
- **Early Error Detection:** Identifies compilation, testing, and validation failures .

4. SYSTEM REQUIREMENTS

1. Hardware Requirements

Processor (CPU)

- Minimum: An Intel Core i3 or AMD Ryzen 3 (or equivalent) processor is required to run the application effectively.
- Recommended: For better performance under heavy load and with larger datasets, an Intel Core i5/i7 or AMD Ryzen 5/7 (or equivalent) is strongly advised.

RAM (Memory)

- Minimum: 4 GB of RAM is required for basic operations.

Hard Disk Storage (HDD/SSD)

- Minimum: 100 GB of available storage space is needed. Recommended: 250 GB or more.

Monitor

- Minimum Resolution: The display should support a resolution of at least 1024x768.
- Recommended Resolution: Full HD (1920x1080) provides better clarity.

Network

- A wired or wireless internet connection is mandatory for scenarios involving client-server communication, such as using a web-based or networked system.

Database Server (If Separate)

- For larger organizational deployments, a separate database server can improve scalability and simplify data management.

2. Software Requirements

2.1. Operating System (OS)

- Server Deployment: Windows Server or Linux distributions can be used to host the CI/CD platform and execute build jobs.
- Recommended Server OS: Ubuntu Linux or other supported Linux distributions are preferred.

2.2. CI/CD Automation Tool

- **Available Tools:** Jenkins, GitHub Actions, GitLab CI/CD, and Azure DevOps can be used to implement job chaining and build pipelines.
- **Recommended Tool:** Jenkins is a suitable choice for a standalone project.

5. ALGORITHM

Step 1: Start

Initialize the CI/CD system and load the pipeline configuration.

Step 2: Initialize Jobs

Read all configured jobs, including job names, dependencies, execution commands, and required conditions.

Step 3: Check Dependencies

Identify the dependencies of each job and determine which jobs are ready for execution.

Step 4: Select a Job

Select a job whose dependencies have been completed successfully and whose execution conditions are satisfied.

Step 5: Execute the Job

Execute the selected job, such as source code checkout, compilation, testing, code analysis, or deployment.

Step 6: Monitor Execution

Monitor the job status and collect execution logs, output, and error information.

Step 7: Check Job Status

- If the job succeeds, mark it as completed and make its dependent jobs eligible for execution.
- If the job fails, mark it as failed and prevent dependent jobs from running when their success is required.
- If retry is configured, retry the failed job according to the specified policy.

Step 8: Update the Pipeline

Update job statuses and identify the next eligible jobs. Independent jobs may execute in parallel if sufficient resources are available.

Step 9: Check Completion

If all required jobs have completed, proceed to the final reporting stage. Otherwise, return to Step 3.

Step 10: Generate Report

Display the pipeline execution status, completed jobs, failed jobs, execution time, and relevant logs.

Step 11: Stop

6. SYSTEM IMPLEMENTATION

The implementation of the Job Chaining and Build Pipelines system focuses on automating software development activities, including source code management, building, testing, and deployment.

1. Source Code Management Module

This module retrieves source code from a version control repository such as GitHub or GitLab. It enables developers to maintain version history.

2. Job Configuration and Build Module

This module allows users to define individual jobs, execution commands, dependencies, and conditions. Generates application packages or executable files using suitable build tools.

3. Job Chaining and testing Module

This module connects jobs according to their dependencies. This module executes automated test cases to verify application functionality.

4. Deployment Module

The deployment module publishes or deploys the successfully built and tested application.

5. Monitoring and Reporting Module

This module monitors job execution, records logs, identifies failures, and displays pipeline status. It helps users analyze execution time and troubleshoot errors.

Execution Flow

Source Code → Build → Test → Quality Check → Deploy → Final Report

Each stage starts according to the configured dependencies and conditions. If a required stage fails, the system stops the dependent stages and records the failure.

EXPECTED OUTPUT

The implemented system provides the following results:

- Automated execution of connected jobs.
- Successful build and testing of the application.
- Automatic triggering of dependent jobs.
- Identification and reporting of failed jobs.
- Monitoring of pipeline execution status.
- Reduced manual effort and improved software delivery consistency.

The proposed implementation demonstrates how job chaining and build pipelines can automate repetitive software development tasks, and support reliable CI/CD practices.

7.CONCLUSION

In conclusion, **job chaining and pipelines** are important concepts in operating systems and computer programming that help execute multiple tasks efficiently and systematically. Job chaining allows a series of jobs or commands to be executed one after another in a specific order, where the execution of one job can depend on the successful completion of the previous job. On the other hand, pipelines allow the output of one process to be passed directly as input to another process, making communication between processes easier and faster. Both techniques reduce manual effort, save time, improve productivity, and make better use of system resources. They are widely used in task automation, data processing, software development, and system administration. Job chaining is useful when tasks must follow a particular sequence, whereas pipelines are especially useful when multiple processes need to work together. Although both techniques serve different purposes, they help simplify complex operations and improve the overall performance of a system. Therefore, understanding job chaining and pipelines is essential for managing tasks effectively, automating workflows, and developing efficient and reliable computer systems**.**ng solution for multitasking environments. The project can be further enhanced by incorporating artificial intelligence techniques, adaptive priority scheduling, and real-time task management to achieve even higher efficiency and smarter decision-making in future operating systems.

8. FUTURE ENHANCEMENT

In the future**, job chaining and pipelines** can be improved with advanced technologies to make task execution faster, smarter, and more reliable.

1. **Advanced Automation:** Automate job execution based on schedules, events, and specific conditions to reduce manual work.
2. **Artificial Intelligence Integration:** Use AI and machine learning to predict job failures and optimize task execution.
3. **Cloud Computing:** Integrate cloud platforms to handle large workloads and provide scalable computing resources.
4. **Error Detection and Recovery:** Develop improved mechanisms to identify errors and automatically restart failed jobs.
5. **Real-Time Monitoring:** Provide real-time updates about job status, execution time, and system performance.
6. **Enhanced Security:** Implement authentication, access control, and data encryption to protect jobs and sensitive information.
7. **Parallel Processing:** Execute multiple independent jobs simultaneously to reduce execution time and improve performance.
8. **Resource Optimization:** Manage CPU, memory, and other system resources efficiently during job execution.
9. **User-Friendly Interface:** Develop simple graphical interfaces to create, manage, and monitor job chains and pipelines easily.
10. **Integration with Modern Tools:** Integrate job chaining and pipelines with DevOps tools, workflow management systems, and distributed computing platforms.

9. REFERENCES

1. GNU Project, *Bash Reference Manual – Pipelines*. Available at: https://www.gnu.org/software/bash/manual/html_node/Pipelines

2. Apache Software Foundation, *Apache Airflow Documentation – Core Concepts*. Available at: https://airflow.apache.org/docs/apache-airflow/stable/core-concepts/index.html

3. Apache Software Foundation, *DAGs – Workflow Scheduling and Task Dependencies*. Available at: https://airflow.apache.org/docs/apache-airflow/stable/concepts/dags.html

4. Kubernetes, *Jobs – Running Tasks to Completion*. Available at: https://kubernetes.io/docs/concepts/workloads/controllers/job/

5. Kubernetes, *Run Jobs – Parallel Processing and Automated Tasks*. Available at: https://kubernetes.io/docs/tasks/job/

6. GNU Project, *Bash Reference Manual*. Available at: https://www.gnu.org/s/bash/manual/bash.html

7. Apache Software Foundation, *Apache Airflow – Task Retries and Workflow Management*. Available at: https://airflow.apache.org/docs/apache-airflow/stable/core-concepts/index.html